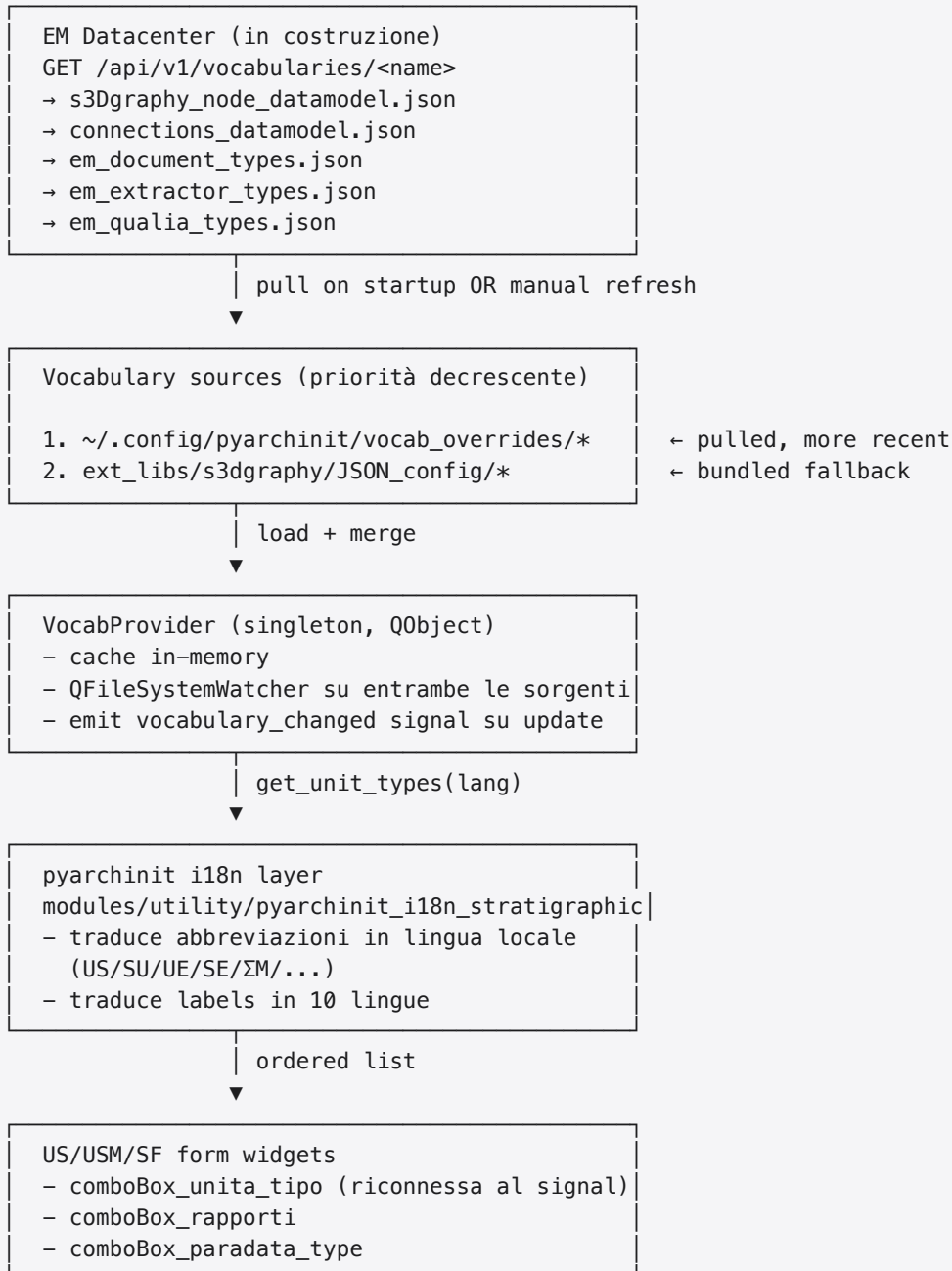


Sezione 3/6 — Vocabulary sync layer

Come i 4 dizionari di s3dgraphy diventano la sorgente delle combobox di pyarchinit

3.1 Catena di propagazione

Da datacenter al widget — chi legge cosa



3.2 API del VocabProvider

```
class VocabProvider(QObject):
    """Singleton. Emette vocabulary_changed quando una sorgente cambia."""

    vocabulary_changed = pyqtSignal(str) # 'units' | 'edges' | 'paradata' | 'cidoc'

    def __init__(self, plugin_dir: Path, user_config_dir: Path) -> None: ...

    # --- Lettura ---
    def get_unit_types(self, lang: str | None = None) -> list[UnitType]:
        """Lista ordinata. UnitType(abbreviation, label, family, description, cidoc_uri)."""

    def get_edge_types(self, lang: str | None = None) -> list[EdgeType]:
        """EdgeType(name, label, canonical_direction, cidoc_uri)."""

    def get_paradata_types(self) -> dict[str, list[ParadataType]]:
        """Raggruppato: 'document', 'extractor', 'combiner', 'author'."""

    def get_cidoc_mapping(self, unit_type: str) -> str | None: ...

    # --- Aggiornamento ---
    def refresh_from_datacenter(self, client: 'DatacenterClient') -> RefreshResult:
        """Pull dei JSON dal datacenter, scrittura in user_config_dir, reload cache.
        Restituisce diff (new/changed/removed) e segnala vocabulary_changed."""

    def reload_local(self) -> None:
        """Force reload (chiamato dal QFileSystemWatcher)."""
```

3.3 i18n: separazione vocabolario / labels

Il datamodel JSON di s3dgraphy ha label inglesi ("label": "US (or SU)"). Pyarchinit serve in 10 lingue. Mantengo la separazione attuale di `pyarchinit_i18n_stratigraphic.py` , ma *cambio chi è la fonte*:

Cosa	Sorgente	Esempio
Quali tipi esistono	s3dgraphy <code>node_datamodel.json</code>	<code>['US', 'USM', 'USVs', 'USVn', 'USD', 'SF', 'TSU', 'UL', 'VSF']</code>
Abbreviazione localizzata	pyarchinit <code>vocab_i18n.json</code>	<code>{ "US": {"it": "US", "en": "SU", "de": "SE", "es": "UE", ...} }</code>
Label e descrizioni	pyarchinit <code>vocab_i18n.json</code>	<code>{ "USVs": {"it": "US Virtuale Strutturale", "en": "Structural Virtual SU"} }</code>
Mapping CIDOC-CRM	s3dgraphy datamodel	<code>"A2 Stratigraphic Volume Unit"</code>
Family / symbol	s3dgraphy datamodel	<code>"family": "real", "symbol": "white rectangle"</code>

Quando s3dgraphy aggiunge un nuovo tipo (es. `WorkingUnit` in 0.1.35), il VocabProvider lo espone subito; pyarchinit fa fallback a label inglese se manca traduzione, e logga warning per chiedere al traduttore di completare `vocab_i18n.json` .

3.4 Hot-reload nelle form aperte

Lo snippet attuale in `US_USM.py:17207` già fa `blockSignals → clear → addItem → setEditText(current_text)`. Lo riusciamo:

```
# In US_USM.__init__(self, ...)
vocab_provider.vocabulary_changed.connect(self._on_vocab_changed)

@pyqtSlot(str)
def _on_vocab_changed(self, vocab_name: str) -> None:
    if vocab_name == 'units':
        self._repopulate_unita_tipo() # già esiste come customize_GUI
    elif vocab_name == 'edges':
        self._repopulate_rapporti_combo()
    # ...
```

Comportamento atteso: lo scavatore ha la scheda US aperta, riceve un push del datacenter (o un nuovo plugin update), il combo si ripopola **senza perdere il valore corrente se ancora valido**. Se il valore corrente non è più valido (es. `USVA` rimosso post-migrazione), si mostra un warning a piè di scheda con suggerimento di sostituzione.

3.5 Refresh policy

Quando	Cosa fa
Plugin start	VocabProvider carica cache locale (~istantaneo). In background lancia <code>refresh_from_datacenter()</code> con timeout 5s. Se fallisce → silent, usa cache.
Pulsante "Sync vocabularies" nel menu PyArchInIt	Refresh esplicito con dialog di feedback (nuovi tipi trovati, deprecati, conflitti).
QFileSystemWatcher rileva modifica al JSON	Reload immediato + emit <code>vocabulary_changed</code> .
Plugin update (s3dgraphy versione bumpata)	I JSON bundled cambiano → watcher li rileva → reload automatico.

NOTE DI IMPLEMENTAZIONE

- Il `vocab_i18n.json` di pyarchinit vive in `resources/vocab/vocab_i18n.json`, modificabile dai traduttori senza toccare codice Python.
- Lo schema di `vocab_i18n.json` usa gli stessi `abbreviation` di s3dgraphy come chiavi: questa è la "join key".
- Se s3dgraphy rimuove un tipo, pyarchinit non lo espone più ma **non cancella i record esistenti** che lo usano. Mostra warning + suggerisce migrazione.
- Per il GraphDB target, il `get_cidoc_mapping` alimenta direttamente l'export RDF — niente lookup runtime separato.